Bibliography
P2P File Synchronization System

Action Learning Project

Backend (Go, C++)

Arihant Jain
Su Huizhe

Frontend and CI/CD (Java, GitHub Actions)

Nizar Soufiya
Shengkang Duan

May 15, 2026

Contents

1 Problem Context

1.1 Scenario and Pain Points

To motivate the problem, we begin with a concrete collaborative scenario commonly observed in university projects.

University students frequently work in temporary groups and need to exchange files efficiently without relying on complex infrastructure, cloud account management, or external storage devices. During the course of a project, team members collaboratively produce and modify heterogeneous files, including source code, documents, images, datasets, and presentation materials. Since collaborators often use different operating systems and development environments, cross-platform compatibility further increases the complexity of file sharing and coordination.

In practice, temporary student teams usually lack the time, resources, and technical expertise required to maintain a dedicated file management or synchronization system. As a result, files are often exchanged through ad hoc communication channels such as group chats or email attachments. This approach introduces several issues, including duplicated files, inconsistent versions, lack of synchronization, and absence of proper change tracking.

In addition, ownership and persistence of project data become problematic once the collaboration ends. Some members may wish to preserve the project files for future reference, while others may prefer to discard them. However, because the files are typically distributed informally and managed inconsistently, long-term access often depends on a single individual maintaining the archive. This creates risks of data loss, incomplete project history, and fragmented ownership.

Therefore, the students require a lightweight and decentralized solution that enables them to:

- create a shared repository without complicated server deployment or administrative setup;
- support synchronization of heterogeneous file types across multiple platforms;
- track modifications and maintain version history;
- provide shared ownership of project data among collaborators; and
- avoid dependence on a single participant for long-term storage, archival, or deletion of the project files.

1.2 Problem Definition

Based on the scenario described above, small-scale collaborative groups require a file-sharing mechanism that is lightweight, decentralized, and easy to use. In such environments, participants mainly care about whether they can quickly form a shared workspace, exchange project files reliably, preserve previous versions when changes are made, and continue working with the shared folder as if it were a normal local directory.

From this objective, the problem can be defined as the design of a peer-to-peer directory-level synchronization system with lightweight version control. More specifically, the system must solve the following technical problems:

- **Local peer discovery and dynamic membership.** The system must discover available peers on the same local network without relying on a central server. It should also support dynamic participation, where peers may join, leave, or temporarily disconnect during collaboration.

- **Directory-level change detection and propagation.** The system must monitor a shared local directory and detect file creation, modification, and deletion events. These changes should be propagated to other online peers without requiring users to manually upload, download, or explicitly send files.
- **Rapid synchronization among online peers.** When a change occurs, the system should synchronize the updated state to other available peers within a short delay, so that collaborators can observe a consistent project workspace during active collaboration.
- **Support for heterogeneous file types and large files.** The system must handle different categories of project files, including text files, source code, documents, images, and binary files. For large files, the system should support efficient transfer mechanisms rather than assuming that every file can be sent as a single small message.
- **Simple versioning and rollback.** Before a file is overwritten by a synchronized update, the system should preserve an older copy or historical state. This provides a basic recovery mechanism against accidental overwrites, incorrect updates, or unintended file changes.
- **Transparent integration with the local file system.** The shared workspace should behave like a normal directory on the user's computer. Users should be able to create, edit, move, and delete files using their usual tools, while synchronization and version management are handled by the system in the background.

Therefore, the central technical problem is to provide a serverless, peer-to-peer, directory-level synchronization mechanism that supports dynamic local-network collaboration, heterogeneous file transfer, rapid update propagation, and basic version recovery while preserving the simplicity of a normal local folder.

2 Overview of Existing Work

3 Comparison of Approaches

4 Identified Gap

4.1 application

We don't want central server (github/ google docs/good drive/ teams)
Too technical (git)
File sharing services have limited sharing capabilities.
Ownership of the files (google drive)
Limited cloud storage….

4.2 technical

P2p no versioning
Versioning no support for ownership/synchronization…
Heterogeneous file support is not good (for git/google docs/googledrive)
Local sharing not support directory structure and consistency (air drop)

5 Positioning Your Project

6 Research to Application Mapping

- **Zero-configuration peer discovery on LAN**
  - **Research Insight:** mDNS + DNS-SD provide lightweight discovery without a central server in local networks
  - **Application in project**: Implement Go-based peer discovery with automatic advertise/browse behaviour; use WLAN as primary transport.
- **Casual ordering for concurrent updates**
  - **Research Insight:** Logical clocks and event ordering reduce ambiguity when multiple peers edit concurrently.
  - **Application in project:** Track per-file version metadata (timestamps + vector-clock style state) in the coordination layer before resolving conflicts.
- **Consistency strategy under failures and partitions**
  - **Research Insight:** CAP trade-oofs show that perfect consistency is difficult during partition/offline scenarios.
  - **Application in project:** Favor availability and eventual convergence on LAN; use deterministic conflict policy (LWW(Last-Write-Wins) + backup copy) and resync after reconnection.
- **Conflict-handling evolution path**
  - **Research Insight:** CRDTs can avoid some mutual conflict handling but increase design complexity.
  - **Application in project:** Keep CRDT as a future extension; current releases uses simpler LWW(Last-Write-Wins) + version-backup workflow for predictable behavior.
- **Bandwidth-efficient synchronization direction**
  - **Research Insight:** Delta-based transfer (rsync-style) reduces unnecessary data movement for large or partially changes files.
  - **Application in Project:** Start with whole-file transfer for implementation simplicity, then add block-diff optimization for larger files and multi-peer scaling.

7 Key References

8 Conclusion